

Programación 2

Trabajo Práctico

Integrador:

Aplicación Java 1→1 con DAO y JDBC

Dominio Elegido: 9. Producto → CódigoBarras

Alumnos:

Matias Orellana (matias.orellan@gmail.com)

Fernando Gómez (fernando.gomez@tupad.utn.edu.ar)

Nicolas Pagola (nicolaspagola@outlook.com)

Rodrigo Galván (rodrigogalvann1@gmail.com)

Link GitHub: <https://github.com/Nicolasp22/TrabajoFinalIntegrador>

Link Video: <https://youtu.be/S7mpLVC7Dlo>

1. Integrantes y Roles

En esta sección, detallen los 4 integrantes y qué rol principal asumió cada uno en el proyecto.

Matias: Arquitectura y Capa Service.

Fernando: Diseño de Base de Datos y Capa DAO

Nicolas: Desarrollo de Capa de Presentación - Menú

Rodrigo: Diseño de Base de Datos, Pruebas y Documentación

2. Elección del Dominio y Justificación

De los dominios propuestos, se seleccionó la opción 9. Producto → CódigoBarras.

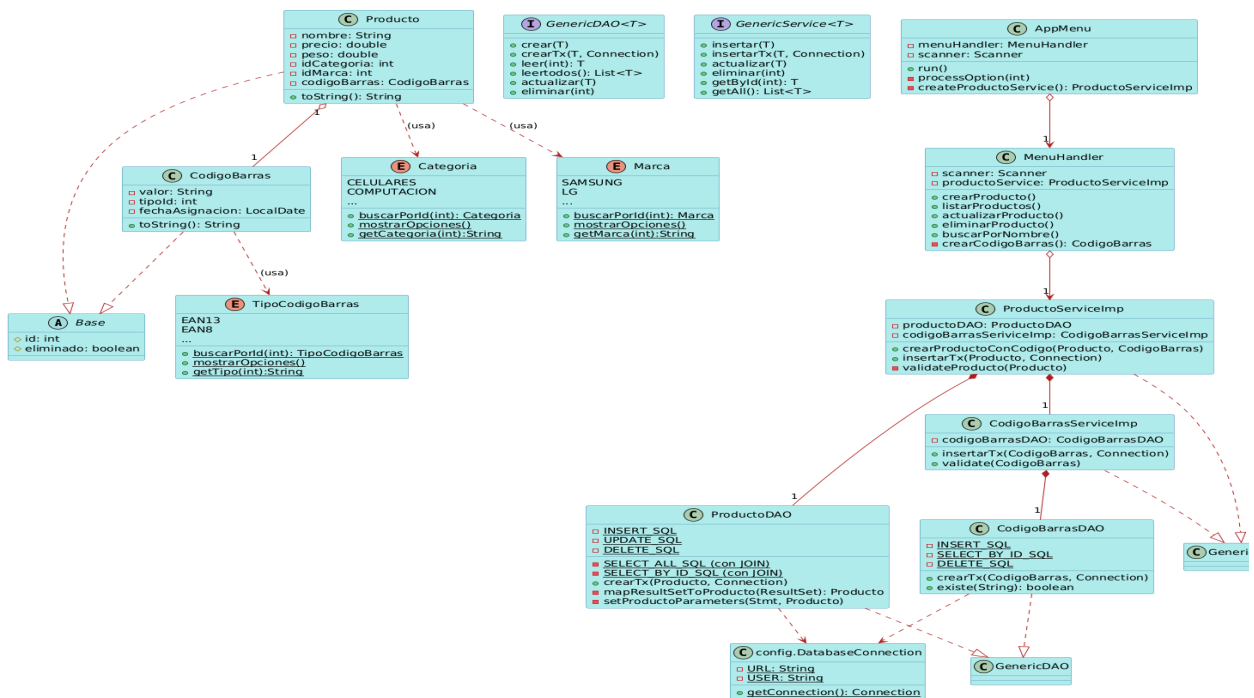
Justificación: La relación entre un Producto y su CódigoBarras representa un caso de uso fundamental en cualquier sistema de inventario, logística o punto de venta (POS). Esta elección permitió modelar una relación 1--1 estricta (un producto tiene un único código de barras, y ese código de barras pertenece a un único producto) que es crítica para la identificación y trazabilidad.

Además, el dominio permitió implementar reglas de negocio claras y relevantes, como la validación de unicidad del valor del código de barras y la gestión de catálogos (Marcas, Categorías) mediante Enums.

3. Diseño y Decisiones Clave

3.1. Diagrama de Clases (UML)

<https://drive.google.com/file/d/1YVaQ449OGmG-aGorGL-il6Ha3IE7JVVM/view?usp=sharing>



El diseño UML sigue la arquitectura de capas propuesta. Destaca la relación unidireccional 1--1 donde la clase Producto (A) mantiene una referencia al objeto CodigoBarras (B), cumpliendo con la especificación del TFI.

3.2. Decisión de Persistencia 1--1

El TFI proponía implementar la relación 1--1 mediante clave foránea única (FK en B) o clave primaria compartida (PK-FK).

Nuestra implementación utiliza una Asociación por Clave Foránea Única, pero con la FK en la tabla A (productos). La tabla codigo_barras (B) tiene su PK (codigo_id). La tabla productos (A) tiene una columna codigo_id que actúa como Clave Foránea (FK) referenciando a codigo_barras(codigo_id).

Para garantizar la cardinalidad 1--1 y evitar 1-*, se aplica una restricción UNIQUE sobre la columna productos.codigo_id.

Esta decisión se tomó para alinear el modelo transaccional, asegurando que el CodigoBarras (B) pueda ser creado primero y su ID generado sea utilizado para crear el Producto (A).

4. Arquitectura por Capas

El proyecto está estructurado en paquetes que separan estrictamente las responsabilidades:

- **config**: Contiene la clase `DatabaseConnection`. Se encarga de gestionar la conexión JDBC con MySQL. Utiliza un bloque estático para "fallar rápido" (fail-fast) si el driver no se encuentra o la configuración es inválida.
- **models**: Contiene las clases que representan las entidades del dominio. Son modelos que reflejan la relación objeto (Producto contiene un objeto `CodigoBarras`). Incluye los Enums (`Marca`, `Categoria`, `TipoCodigoBarras`) para gestionar datos maestros.
- **dao**: Capa de Acceso a Datos. Es la única capa que contiene lenguaje SQL:
 - Implementa el patrón `GenericDAO`.
 - Traduce los `ResultSet` de SQL a los modelos de objetos anidados (Ej: `mapResultSetToProducto` usa `LEFT JOIN` para poblar el `CodigoBarras` interno).
 - Provee métodos transaccionales (ej. `crearTx( T entidad, Connection conn)`) que aceptan una conexión externa para participar de una transacción orquestada por el Service.
- **service**: Capa de Lógica de Negocio.
 - Orquesta las operaciones y valida las reglas de negocio.
 - Es la dueña de las transacciones compuestas.
 - Inyecta dependencias (un Service puede usar varios DAOs, o incluso otros Services).
- **main**: Capa de Presentación (Vista).
 - Contiene el punto de entrada (`Main.java`), el bucle principal y la inyección de dependencias (`AppMenu.java`), y el manejador de la lógica de consola (`MenuHandler.java`). Es "tonta" y solo delega las llamadas al Service.

5. Persistencia y Transacciones

La persistencia se realiza con JDBC puro, utilizando `PreparedStatement` para evitar inyección SQL.

La gestión transaccional es el núcleo de la capa de servicio, como se evidencia en la operación de crear un nuevo producto (`ProductoServiceImp.crearProductoConCodigo`):

Apertura de Conexión: El Service obtiene una única `Connection` de `DatabaseConnection.getConnection()`:

- Inicio de Transacción: Se deshabilita el modo `autoCommit` llamando a `conn.setAutoCommit(false)`.
- Operación Compuesta:
 - Se valida el `CodigoBarras` (ej. unicidad).

- Se invoca a `codigoBarrasServiceImp.insertarTx(codigoBarras, conn)`, pasando la conexión.
- Se obtiene el ID generado del `codigoBarras` y se asigna al objeto `Producto`.
- Se invoca a `productoDAO.crearTx(producto, conn)`, pasando la misma conexión.
- Confirmación (Commit): Si ambas operaciones (INSERT en `codigo_barras` e INSERT en `productos`) son exitosas, el Service ejecuta `conn.commit()`. Finalmente se reestablece el `conn.setAutoCommit(true)`, para devolver la conexión a un estado normal practicando para cuando trabajemos con Connection Pooling.
- Reversión (Rollback): Si ocurre cualquier `SQLException` (ej. validación de BD, unicidad, etc.), el bloque catch captura la excepción y ejecuta `conn.rollback()`, asegurando que la base de datos vuelva a su estado anterior y no se generen datos inconsistentes.
- Cierre: Finalmente, la conexión se cierra, restableciendo el `autoCommit` (implícito en el `try-with-resources`).

6. Validaciones y Reglas de Negocio

Las validaciones se implementan en la capa de Service para lógica de negocio y en la capa de Models para datos maestros.

Validación de Unicidad (BD): El `CodigoBarrasServiceImp` utiliza un método `codigoBarrasDAO.existe(valor)` para consultar la BD y asegurar que un código de barras no pueda ser duplicado antes de intentar la inserción.

- Validación de Formato: Se valida la longitud mínima del código de barras (`valor.length() < 8`).
- Validación de Negocio: Se valida que los valores numéricos sean positivos (ej. `precio > 0` en `MenuHandler` o `Service`).
- Integridad de Datos (Enums): El ingreso de `idCategoria` e `idMarca` se valida usando los métodos estáticos `Categoria.buscarPorId(id)` y `Marca.buscarPorId(id)`, que lanzan una `IllegalArgumentException` si el ID no existe.
- Baja Lógica: La eliminación de productos se implementa como baja lógica (`UPDATE productos SET eliminado = TRUE`), preservando la integridad referencial.

7. Pruebas Realizadas

Para garantizar la robustez de la aplicación, se desarrollaron clases de prueba en el paquete `Tests`:

- `TestConexion.java`: Una prueba de "humo" simple para verificar que la configuración de `DatabaseConnection` es correcta y que la conexión a MySQL es exitosa.
- `TestProductoServiceImp.java`: Una prueba de integración que valida el componente más crítico: el método transaccional `crearProductoConCodigo`. Prueba la lógica de Service y DAO directamente contra la base de datos.

- TestMenuHandlerSimple.java: Una suite de pruebas de flujo completo (E2E) que simula la entrada del usuario en la consola mediante ByteArrayInputStream. Esta suite valida:
 - El flujo completo de creación.
 - La correcta eliminación del N+1 en la lectura (listarProductos).
 - La respuesta del sistema a entradas inválidas (precio negativo, ID de categoría inexistente, código de barras corto).

```

===== GESTION DE PRODUCTOS =====
1. Crear Producto y asociarCodigo de Barras (Transaccional)
2. Listar todos los Productos (incluyendo baja logica)
3. Buscar producto por Id
4. Actualizar Precio de un Producto
5. Eliminar Producto (Baja Logica)
6. Limpiar Consola
--- Busquedas ---
7. Buscar Producto por nombre (Busqueda especial)
0. Salir
Ingrese una opcion: 2
Id Producto : 2
Producto    : Producto Test
Precio      : $100,00
Peso        : 0,50g
Categoria   : Computacion
Marca       : Samsung
Codigo
Id Codigo   : 2
Valor       : 1234567890123
Fecha       : null
Observaciones: null
Tipo        : EAN-13
=====
Id Producto : 3
Producto    : Lavarropas
Precio      : $450,00
Peso        : 50,00g
Categoria   : Linea Blanca
Marca       : Samsung
Codigo
Id Codigo   : 3
Valor       : lavarropas123
Fecha       : null
Observaciones: null
Tipo        : EAN-13
=====

```

```

=== TEST SIMPLE MENU HANDLER ===

1. Probando constructor...
? OK: Constructor detecta scanner null - Scanner no puede ser null

2. Probando crear codigo de barras...
Valor del codigo de barras(minimo 8 digitos): Tipos de codigo de barras di
1. EAN-13
2. UPC
3. Code 128
Seleccionar tipo: ? OK:Codigo creado - Valor: 1234567890123, Tipo: 1, id

3. Probando listar productos (simulado)...
Id Producto : 2
Producto    : Producto Test
Precio      : $100,00
Peso        : 0,50g
Categoria   : Computacion
Marca       : Samsung
Codigo
Id Codigo   : 2
Valor       : 1234567890123
Fecha       : null
Observaciones: null
Tipo        : EAN-13
=====

Ocurrio un error no-SQL.
Error al crear producto: El codigo de barras debe tener al menos 8 digitos
Seleccionar tipo: ? Codigo muy corto - Debio fallar
Error al crear producto: El nombre no puede estar vacio.
Nombre: ? Nombre vacio - Debio fallar
Error al crear producto: El peso debe ser mayor a 0.
Nombre: Precio: Peso: ? Peso negativo - Debio fallar
=== FIN TESTS ===
BUILD SUCCESSFUL (total time: 1 second)

```

- Test de Rollback.

Se realiza nuevamente el test TestMenuHandlerSimple.java sin modificar el valor del código de barras, de esta manera cuando el dao envía el insert a la base, el constraint de la misma base de datos no permite la inserción duplicada de valor y devuelve una SQLException. Esto debería evitar la construcción de todo el producto y hacer un rollback. Para que funcione comentamos **existe(String valor)** en el codigoBarrasDAO para que la validación no frenara la ejecución y llegara hasta la capa de sql.

8. Conclusiones y Mejoras Futuras

El desarrollo de este TFI permitió implementar los conceptos fundamentales de la programación de aplicaciones de 3 capas. El mayor desafío y logro fue la correcta

implementación de la arquitectura DAO/Service, el manejo de la persistencia con JDBC puro y, fundamentalmente, la orquestación de transacciones para operaciones compuestas, asegurando la atomicidad (ACID) de los datos.

Mejoras Futuras:

- Pool de Conexiones: Reemplazar DatabaseConnection con un pool para gestionar eficientemente las conexiones en un entorno multiusuario.
- Frameworks: Migrar la aplicación a un framework como Spring Boot, que abstrae la gestión de transacciones y la persistencia de forma declarativa.

9. Fuentes y Herramientas Utilizadas

- Lenguaje: Java (JDK 21)
- IDE: Apache NetBeans
- Base de Datos: MySQL
- Driver JDBC: [com.mysql.cj.jdbc.Driver](#)
- BDMS: MySQL WorkBench.
- Asistencia IA: Se utilizó el asistente de IA Gemini para consultas de sintaxis, depuración de errores (N+1, NoSuchMethodError) y análisis de arquitectura (debate de patrones), cumpliendo con la solicitud de citar su uso.